Course Project Guideline

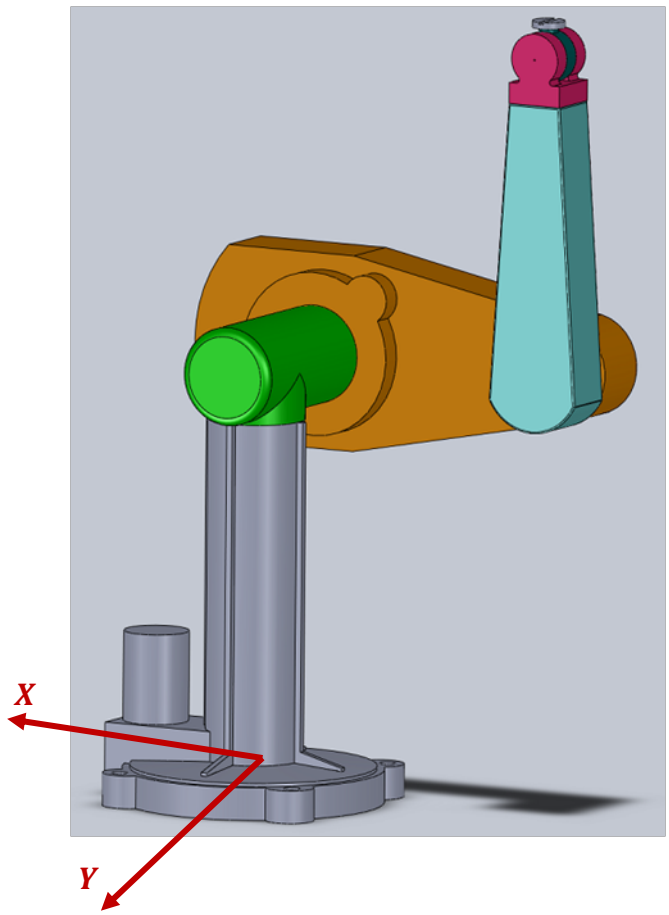
Project Option 1: Robot Simulation in Matlab

This project is tailored to facilitate the learning basic concepts of kinematics, dynamics and control of robot manipulator. You will develop a virtual PUMA560-type robot simulator in Matlab. The project is divided into 3 phases in synchronize with course progress. **The first phase** requires students to model a PUMA560-type of 6 degrees-of-freedom robot using Solidworks. Details such as drive systems or electrical wiring are not necessary for this CAD model. This model with assembly of each link will be imported into MATLAB as patch objects. **The second phase** is to develop a graphical user interface which displays the robot configuration as 3D model, and provides options of demonstrating independent joint motion (forward kinematics) and trajectory following of end-effector (inverse kinematics). **The third phase (Bonus)** is to develop a PID controller for each joint or a model-based nonlinear controller. Details for each step are given below.

- Step 1: Create your own robotic manipulator model.

This robot model has to contain 6 degrees-of-freedom with a spherical wrist. You can create your own model using Solidworks, AutoCad Inventor or any other CAD software. The CAD model doesn't have to be fancy as long as it contains properly assembled parts and gives 6 degrees-of-freedom. If you are not interested in solid modeling, you can also download a CAD model from industrial robot manufacturer's website such as ADEPT, FANUC, KUKA, DESO, etc. Generally, manufacturers provide CAD model for commonly used software. Choose a robot model with detailed documentation.

For example, a PUMA560 (not exact shape) was created using SolidWorks as shown below.



(Figure 1 Simplified PUMA560 CAD model)

Here are few things to note:

(1) When model the link 1 (base of the robot), make sure that the origin of the coordinate system for modeling coincides with the center of the bottom surface of link 1. And the bottom surface should be sketched in the "front plane" or XY-planes. Then the direction of extrusion

of the link 1 is in the positive direction of z-axis. In actual PUMA560, the world frame has X-axis going through the joint 1 motor. Although for simulation, one can place X-axis in any directions.

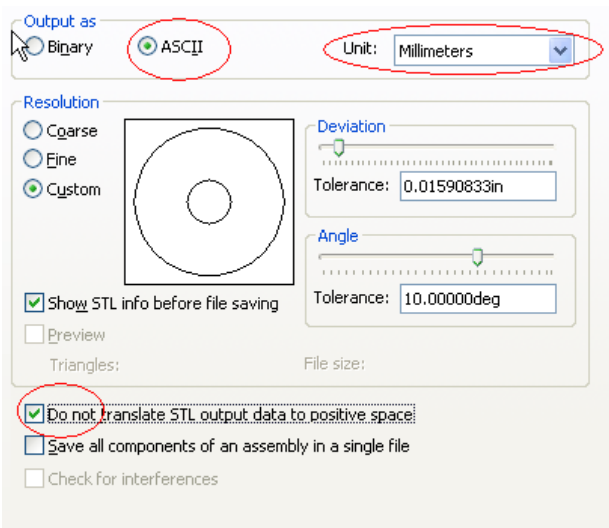
(2) When assembling all links, makes sure to use constrain condition to have the origin of part 1 (link 1 module) coincides with the origin of the coordinate system in the assembly space. You may need to "float" the part 1 first, apply the constraint and fix the part 1 in the end.

(3) Using constraints to assemble the parts of the robot into the "READY" pose of the robot. PUMA560's "READY" pose is shown in Figure 1, which is also the "zero position" (all joint angles are zero degree). In the next step, when we export the assembly to STL file. The STL files will contain the actual XYZ position information for each part in the "READY" pose.

- <u>Step 2</u>: Create MAT-files for use in Matlab

Firstly, save the assembly of your robot model as "STL" files in ASCII format. Quoting from Wikipedia, "STL is a file format native to the stereolithography CAD software created by 3D Systems. This file format is supported by many other software packages; it is widely used for rapid prototyping and computer-aided manufacturing. STL files describe only the surface geometry of a three dimensional object without any representation of color, texture or other common CAD model attributes." A STL file is basically a triangular representation of a 3D object. An STL file describes a raw unstructured triangulated surface by the unit normal and vertices (ordered by the right-hand rule) of the triangles using a three-dimensional Cartesian coordinate system [1].

When saving as STL files, select the output format as ASCII and also check the "Do not translate STL output data to positive space" to preserver originality of the data. See the Figure 2 for illustration when saving as STL files in Solidworks.



(Figure 2 Saving options for STL file)

Note that you should have a list of STL files generated depending on the number of parts used in the assembly. These STL files contain geometric information of each part (i.e., X, Y, Z information).

Secondly, convert the STL-files into MAT-file using "Robot_CAD2MAT.m". This m-file is a modified based on "cad2matdemo.m" which can be found on Matlab central (http://www.mathworks.com/matlabcentral/fileexchange/3642-cad2matdemo-m).

Open file "Robot_CAD2MAT.m" and fill in the file names as your designated file names.

"Robot_CAD2MAT.m" reads each STL-file (corresponding to each link) and extracts data of faces (F), vertices (V), and colors (C) into three variables F_i, V_i, and C_i, respectively. The original size of $V1$ is $n \times 3$ where n is the number of vertices, 3 is the number of coordinates for

each vertex (x, y, z). For homogeneous transformation, we need to add 1 to make it $(x, y, z, 1)$. Thus, we concatenate a column of 1 with n rows to matrix V_i to make it a $n \times 4$ matrix. Then the m-file saves these three variables F_i, V_i, and C_i, into a "struct" variable. For example, for link 1, we can have

```
s1=struct('F1', F1, 'V1', V1, 'C1', C1);
```

Every link or part in the robot model will be represented using one "struct" variable. All "struct" variables will be saved into a MAT-file.

```
save('your_robot_data.mat', 's*');
```

Now this MAT-file can be loaded in the main simulation file and a "patch object" can be created corresponding to each link using the Matlab command "patch". For example

```
L1=patch('faces', s1.F1, 'vertices', s1.V1(:, 1:3));
```

The patch object will be used for computer visualization in Matlab environment. In the next few steps, one can modify the value of vertices by applying rotation/translation to corresponding variables (e.g., $V1$). Animated motion of the robot can be easily realized.

If you don't like the current naming conventions, feel free to change variable names.

✓ About the "patch" command

"patch" command [2] is a convenient tool to create 3D graphics, which is built on window open GL. A sample usage of this command is shown below.

```
L1=patch('faces', s1.F1, 'vertices', s1.V1(:, 1:3));
```

- "vertices", s1.V1(:, 1:3) is a $n \times 3$ matrix that contains all the vertices used to describe the link 1. n is the number of vertices, 3 is the number of coordinates for each vertex (x, y, z). Each row of this matrix corresponds to a vertex and each column to its x, y, z coordinates.
- 'faces': $F1$ is a matrix that defines each face of the object in terms of the vertices that lie on the face. Each row of this matrix corresponds to a face, the column element of each row refers to the vertices that define the face.
- "L1" is a patch object created. We can use "set" command to change its properties and update the values of vertices after rotation/translation. For example,

  set(L1, 'facec', [0.717,0.116,0.123]); %setting face color

  set(L1, 'EdgeColor','none'); % setting edge color of the faces

- <u>Step 3</u>: Create an interface using Matlab GUI

Type "guide" to enter the Matlab GUI design environment. One can drag and drop various components (button, textbox, axis etc.) to create unique simulation user interface. The GUI should include the following functionalities

- ✓ Demonstration forward kinematics, inverse kinematics and at least one control implementation.
- ✓ For inverse kinematics, show the end-effector trajectory in real-time.
- ✓ Real-time outputs of current joint angles and end-effector's position and orientation are preferred. The orientation can be represented using Euler's angle (ZYZ).

- <u>Step 4</u>: Implement simulation of forward kinematics
  - ✓ Obtain dimensions of your robot model
  - ✓ Assign frames to the robot using DH convention and generate DH table.
  - ✓ Realize the homogenous transformation matrix in Matlab based on the DH table
  - ✓ Implement forward kinematics and demonstrate joint motion

How to use homogeneous transformation matrix?

- ✓ Define matrices Link0, Link1, Link 2, …, Link_n to contain all the vertices information for each link. Columns of these matrices are the homogeneous coordinates of vertices. Since Link0 is fixed, no change will be applied to this matrix.
- ✓ The frame 0 is designated as world frame. Homogeneous coordinates of vertices are all with respect to the world frame.
- ✓ Effect of rotation about joint 1 or rotation about z_0 (θ_1).

  Link1 to Link6 will all rotate about z_0. Thus rotates vertices on each link will be

  $$T_z(\theta_1) \cdot \text{Link1}, \dots \dots, T_z(\theta_1) \cdot \text{Link2}$$

  $T_z(\theta)$ is the basic homogeneous transformation matrix for pure rotation about the current z-axis by an amount of θ.
- ✓ Effect of rotation about joint 2 or rotation about z_1 (θ_2).

  Link2 to Link_n will all rotate about z_1. What we have is the homogeneous coordinates with respect to frame 0. Since this rotation about z_1 is with respect to current frame, we need to first transform the homogeneous coordinates of Link2 to Link_n into frame 1. Then apply the pure rotation about the current z_1-axis and finally transform back to frame 0. The overall operation is

$$T_1^0 \cdot T_z(\theta_2) \cdot (T_1^0)^{-1} \cdot \text{Link}_i, \qquad i = 2, \dots n$$

- ✓ Effect of rotation about joint 3 (frame 2) or rotation about z_2 (θ_3).

  Link3 to Link6 will all rotate about z_2. Similarly we need to first transform the homogeneous coordinates of Link3 to Link6 into frame 2. Then apply the pure rotation about the current z_2-axis and finally transform back to frame 0. The overall operation is

$$T_2^0 \cdot T_z(\theta_3) \cdot (T_2^0)^{-1} \cdot \text{Link}_i, \qquad i = 3, \dots n$$

- ✓ In general, the effect of rotation about joint i (frame $i-1$) or rotation about z_{i-1} (θ_i).

  Link i to Link6 will all rotate about z_{i-1}.

$$T_{k-1}^0 \cdot T_z(\theta_k) \cdot (T_{k-1}^0)^{-1} \cdot \text{Link}_k, \qquad k = i, \dots n$$

- ✓ Update the vertices of the patch objects for each link

- <u>Step5</u>: Implement simulation of inverse kinematics
  - ✓ Generate a suitable end-effector trajectory
  - ✓ Implement inverse kinematics for each joint based on what was taught in class.
  - ✓ Demonstrate the motion of the robot in Matlab and plot the end-effector trajectory.

- <u>Step 6</u>: Implement a control technique
  - ✓ Dynamic model of the robot is required in order to implement the controller and simulate the controlled motion. Even if it is a non-model based controlled (e.g., PID control), one still needs the dynamic model for simulation. In view of the complexity

of deriving dynamic equations of motion for a 6 degrees-of-freedom robot, you have an option here.

Option 1: if you would like to take on the challenge, you can use Lagrange formulation to derive the equations of motion. Here is a hint. Generally, CAD software will provide moment of inertia about the center of mass for each part. Also, for symbolic formulation, one can use either Maple or Matlab.

Option 2: Use a simplified robot model. You can consider the 2nd and 3rd links as a two-link **planar** manipulator with uniform linkages. Derive the equations of motion using Lagrange's formulation for this two-link planar manipulator.

✓ Controller implementation and simulation

Option 1: independent joint controller such as PID or PD control.

Option2: model based control such as computer torque control

- Step 7: Writing of report

Final report should be in article format, maximum 30 pages, single columns, 1.5 spaced, 12 point font, including figures and references. Major papers referenced should be attached as an appendix.

Reference

[1] STL (file format), From Wikipedia.

[2] Matlab manual.